Solució · Repàs de POO: l'agroturisme Son Ametler

DWES · 2n DAW · Tema 1 (POO) · Curs 2026-27

Aquí hi ha les respostes de totes les preguntes, numerades com a l'enunciat, i les parts importants del codi. El codi complet i comentat és a `solucio-practica.zip`, amb una carpeta per a cada part (de la 2 a la 6) tal com queda el codi en acabar-la. A IntelliJ, *File > Open*, triau el `pom.xml` i *Open as Project*.

> Intentau fer cada part abans de llegir-ne la solució. En les preguntes de raonar, la vostra resposta no ha de ser igual que aquesta: el que compta és que digui el mateix i ho justifiqui.

1 UML: llegir el model

1.1.

		Per què
a)	**Fals**	A l'extrem de `Client` hi ha un `1`: cada reserva és d'un sol client.
b)	**Vertader**	A l'extrem de `Reserva` hi ha `0..*`: un client pot tenir cap reserva o moltes.
c)	**Fals**	A l'extrem d'`Allotjament` hi ha `1..*`: com a mínim un.
d)	**Fals**	A l'extrem de `Factura` hi ha `0..1`: fins que el client no se'n va, la reserva no té factura.
e)	**Fals**	A l'extrem `recomanatPer` hi ha `0..1`: com a molt, l'ha recomanat un client.

> El número que diu «quants B té cada A» s'escriu a l'extrem de `B`, el més allunyat d'`A`. És l'error més freqüent en llegir diagrames.

1.2. `Agroturisme` – `Allotjament` i `Reserva` – `Activitat`: són les que tenen el rombe negre, a la banda del tot. En una composició, les parts són exclusives del tot i no existeixen sense ell: si s'esborra una reserva, les seves activitats contractades s'esborren també.

1.3. La cursiva vol dir **abstracte**. `Allotjament` és una classe abstracta: no se'n poden crear objectes (`new Allotjament(...)` no compila), perquè tot allotjament és una habitació, una suite o una casa. `preuNit()` és una operació abstracta: no té cos al pare, i cada filla l'ha d'implementar a la seva manera.

1.4. Una classe associativa guarda els atributs d'una **parella** d'objectes units: la puntuació no és ni del client (en fa moltes) ni de l'allotjament (en rep moltes), sinó de «aquest client valorant aquest allotjament». Per això n'hi ha **un sol objecte per a cada parella**, i l'enunciat ho permet: un client valora un allotjament una sola vegada. Una reserva, no: l'Aina i la Garrover tenen dues reserves, la de l'agost i la del novembre, i una classe associativa no pot tenir dos objectes per a la mateixa parella. Per això `Reserva` és una classe normal, amb dues associacions.

1.5. Perquè es calculen a partir d'altres dades: les nits, de les dates, i el preu, de les nits i de l'allotjament. Si fossin atributs, la mateixa informació seria a dos llocs i podria deixar de quadrar. És el que passa al codi del becari (part 2): canvia la sortida i les nits no.

1.6. Cada fletxa que surt de `Reserva` és un atribut de `Reserva`. Si la cardinalitat de l'altre extrem és `1`, l'atribut és un objecte; si és `0..*`, una llista.

```
public class Reserva {
    private String codi;
    private LocalDate entrada;
    private LocalDate sortida;
    private int persones;
    private boolean cancelada;
    private List<String> observacions;
    private Client client;                      // Reserva → Client, amb 1
    private Allotjament allotjament;            // Reserva → Allotjament, amb 1
    private List<Activitat> activitats;        // Reserva ◆↔ Activitat, amb 0..*
}
```

L'associació amb `Factura` no afegeix cap atribut a `Reserva`: la fletxa va de `Factura` cap a `Reserva`, i per tant és `Factura` qui té l'atribut `reserva`. Al codi de la pràctica ho simplificam una mica: les activitats no es guarden dins la reserva, sinó que van directament a la factura (part 5).

2 Encapsulació: el codi del becari

2.1. Totes dues classes segueixen el model de `Client`: atributs `private`, `final` els que no canvien, un getter per a cada dada que s'ha de poder consultar, i setters només on l'enunciat diu que es pot canviar. En tot el programa només hi ha tres setters: `setNom`, `setEmail` i `setPreuBase`. La resta de canvis es fan amb **operacions amb significat** (`cancelar`, `afegirObservacio`), que comproven les regles.

```
public class Habitacio {
    public static final double SUPLEMENT_VISTES = 15;
    public static final int PLACES_MAXIMES = 4;

    private final String codi;
    private final String nom;
    private final int places;
    private double preuBase;                        // no és final: es pot canviar amb
setPreuBase
    private final boolean vistes;

    // constructor que ho comprova tot, getters, teVistes() i toString()

    public void setPreuBase(double preuBase) {
        this.preuBase = preuPositiu(preuBase);      // la mateixa comprovació que el
constructor
    }

    public double preuNit() {                       // el suplement es calcula aquí, i
amb una constant
        return vistes ? preuBase + SUPLEMENT_VISTES : preuBase;
    }

    private static double preuPositiu(double preu) {
        if (preu <= 0) {
            throw new IllegalArgumentException("El preu ha de ser més gran que 0");
```

```
        }
        return preu;
    }
}
```

```
public class Reserva {
    private static int comptador = 0;                // de classe: un per a totes les
reserves

    private final String codi;
    private final Client client;
    private final Habitacio habitacio;
    private final LocalDate entrada;
    private final LocalDate sortida;
    private final int persones;
    private boolean cancelada;                       // comença a false
    private final List<String> observacions = new ArrayList<>();   // i cap atribut nits
ni preu

    public Reserva(Client client, Habitacio habitacio, LocalDate entrada, LocalDate
sortida, int persones) {
        if (client == null || habitacio == null) {
            throw new IllegalArgumentException("El client i l'habitació són obligatoris");
        }
        if (persones < 1 || persones > habitacio.getPlaces()) {
            throw new IllegalArgumentException("Hi poden anar entre 1 i " +
habitacio.getPlaces() + " persones");
        }
        comprovarDates(entrada, sortida);
        this.client = client;
        // ... la resta d'atributs ...
        comptador++;                                  // AL FINAL: una reserva rebutjada
no gasta cap número
        this.codi = String.format("R%03d", comptador);
    }

    public List<String> getObservacions() {
        return Collections.unmodifiableList(observacions);   // de només lectura
    }

    public void afegirObservacio(String text) {
        if (text == null || text.isBlank()) {
            throw new IllegalArgumentException("L'observació no pot ser buida");
        }
        observacions.add(text);
    }

    public void cancelar() {
        if (cancelada) {                              // el valor és correcte, però
l'objecte no pot fer-ho
            throw new IllegalStateException("La reserva " + codi + " ja està
cancel·lada");
        }
        cancelada = true;
    }

    public int nits() {                               // es calcula: sempre quadra amb les
dates
        return (int) ChronoUnit.DAYS.between(entrada, sortida);
```

```
    }

    public double preu() {
        return habitacio.preuNit() * nits();          // les vistes les sap l'habitació
    }

    private static void comprovarDates(LocalDate entrada, LocalDate sortida) {
        if (entrada == null || sortida == null) {
            throw new IllegalArgumentException("Les dates són obligatòries");
        }
        if (!sortida.isAfter(entrada)) {            // amb dates no es pot fer servir < ni
>
            throw new IllegalArgumentException("La sortida ha de ser posterior a
l'entrada");
        }
    }
    // getters, estaCancelada() i toString()
}
```

2.2. Totes set deixen de compilar, perquè els atributs ja són privats. Les que es poden fer, ara passen per un mètode que comprova les regles:

#	Línia del becari	Ara
1	`r.sortida = …;`	No es pot fer: les dates d'una reserva no canvien. Les nits i el preu ja no són atributs, i sempre quadren amb les dates.
2	`r.persones = 5;`	No es pot fer: `final` i sense setter. I el constructor no accepta més persones que places.
3	`r.codi = "R999";`	No es pot fer: el codi el posa el sistema.
4	`Reserva.comptador = 0;`	No es pot fer: el comptador és privat.
5	`garrover.preuNit = -90;`	`garrover.setPreuBase(-90)`: compila, però llança `IllegalArgumentException`.
6	`r.cancelada = true;` / `false`	`r.cancelar()`. Si ja està cancel·lada, llança `IllegalStateException`. No hi ha cap manera de «descancel·lar».
7	`r.observacions.clear();`	`r.getObservacions().clear()`: compila, però llança `UnsupportedOperationException`.

2.3. La sortida és la de l'enunciat. Fixau-vos també en el **nom de l'excepció** de cada `[OK]`: si en surt una `NullPointerException`, el codi ha fallat per casualitat, sense comprovar res. Passa, per exemple, si es comprova `persones > habitacio.getPlaces()` abans que `habitacio` no sigui `null`.

2.4.

a) Perquè es calculen a partir de les dates i de l'habitació. Si són atributs, la informació és a dos llocs i pot deixar de quadrar. El `Main` del becari ho demostra: després d'allargar l'estada fins al dia 20, continua dient «3 nits, 315.0 €».

b) `static`, perquè hi ha d'haver **un sol comptador per a totes les reserves**: és de la classe, no de cada objecte. Si fos un atribut d'objecte, cada reserva tendria el seu, començaria a 0 i totes serien la R001.

`private`, perquè ningú de fora el pugui tornar a posar a 0 (la línia 4 del becari) i repetir codis.

c) Perquè el constructor pot llançar una excepció a mitja feina. Si el número es dona al principi, cada reserva rebutjada en gasta un i els codis fan forats. Ho comprova la darrera línia de `ProvaPart2`: després de tres reserves rebutjades, la nova ha de ser la **R003**. Amb el comptador al principi, seria la R006.

3 Herència: les suites

3.1.

```
public class Suite extends Habitacio {
    public static final double RECARREC_SUITE = 0.25;     // un 25 % més que una habitació

    private final int metres;                             // només el que és nou

    public Suite(String codi, String nom, int places, double preuBase, boolean vistes, int
metres) {
        super(codi, nom, places, preuBase, vistes);       // Habitacio ja ho comprova tot
        if (metres <= 0) {
            throw new IllegalArgumentException("Els metres han de ser més que 0");
        }
        this.metres = metres;
    }

    public int getMetres() {
        return metres;
    }

    @Override
    public double preuNit() {
        return super.preuNit() * (1 + RECARREC_SUITE);    // el d'una habitació, vistes
incloses
    }

    @Override
    public String toString() {
        return super.toString() + " · suite de " + metres + " m²";
    }
}
```

La suite hereta d'`Habitacio` les dades, totes les comprovacions (també la de les 4 places) i `setPreuBase`. Per això la prova rebutja una suite de 5 places o amb preu negatiu sense que `Suite` en sàpiga res.

3.2. La sortida és la de l'enunciat. L'Olivera (100 €, amb vistes) val (100 + 15) × 1,25 = 143,75 € per nit.

3.3.

a) `super(codi, nom, places, preuBase, vistes);`. Si la llevau, Java hi posa `super()` sense paràmetres, i no compila (*constructor Habitacio in class Habitacio cannot be applied to given types*): `Habitacio` no té cap constructor sense paràmetres, i la part del pare s'ha de construir primer.

b) No compila: *preuBase has private access in Habitacio*. La suite té el preu base, però no hi pot accedir directament. `super.preuNit()` és millor perquè reaprofita el càlcul del pare tal com sigui: si demà canvia el suplement de vistes, la suite el segueix sense tocar-la. Fer `preuBase` `protected` funcionaria, però trencaria l'encapsulació: les subclasses podrien posar-hi un preu negatiu sense passar per la comprovació.

c) Compila, perquè `preuNIt()` és un mètode nou que no sobreescriu res. Però les suites es cobren com una habitació: la prova escriu 115,00 € per a l'Olivera i 100,00 € per a la Llimonera, sense cap error. Amb `@Override`, el compilador ho detecta (*method does not override or implement a method from a supertype*): per això s'ha de posar sempre.

3.4.

	Compila?	Què passa, o per què no
a)	**Sí**	Una `Suite` és una `Habitacio`: la variable del pare pot guardar un objecte de la filla.
b)	**Sí**	Escriu `125.0`. S'executa el `preuNit()` de la suite: es tria pel tipus de l'**objecte**, no pel de la variable.
c)	**No**	*cannot find symbol*. El compilador només mira el tipus de la **variable**, `Habitacio`, que no té `getMetres()`.
d)	**Sí**	Escriu `35`. Amb el càsting, la variable és de tipus `Suite`, i l'objecte ho és de veres.
e)	**No**	*incompatible types: Habitacio cannot be converted to Suite*. Una habitació no és una suite.

4 Classes abstractes: les cases de pagès

4.1.

a) Perquè una casa **no és una** habitació. Heretaria el preu base i les vistes, que no té, i la regla de les 4 places, que rebutjaria Es Pou Vell, de 6.

b) Una **classe abstracta**, perquè:

- les filles comparteixen **dades i codi** (codi, nom, places, les comprovacions, `preuEstada`, `toString`), i una interfície no pot tenir atributs ni constructor;
- no hi ha cap allotjament que no sigui d'un tipus concret, i `preuNit()` no té cap resposta general: ha de ser abstracte;
- hi ha un «és un» clar: una casa és un allotjament.

4.2.

```
public abstract class Allotjament {
    private final String codi;
    private final String nom;
    private final int places;

    protected Allotjament(String codi, String nom, int places) {   // només el criden les filles
        // comprova que el codi i el nom no siguin buits i que hi hagi com a mínim una plaça
    }

    // getCodi(), getNom() i getPlaces()

    public abstract double preuNit();                           // sense cos
```

```
    public double preuEstada(int nits) {
        return preuNit() * nits;                                    // crida un mètode
abstracte
    }

    @Override
    public String toString() {
        return codi + " " + nom + " · " + places + " places";
    }
}
```

```
public class Casa extends Allotjament {
    public static final double PREU_PLACA = 30;
    public static final double SUPLEMENT_PISCINA = 40;
    public static final double NETEJA_FINAL = 60;

    private final boolean piscina;

    public Casa(String codi, String nom, int places, boolean piscina) {
        super(codi, nom, places);
        this.piscina = piscina;
    }

    @Override
    public double preuNit() {                                       // obligatori: és
abstracte al pare
        return getPlaces() * PREU_PLACA + (piscina ? SUPLEMENT_PISCINA : 0);
    }

    @Override
    public double preuEstada(int nits) {
        return super.preuEstada(nits) + NETEJA_FINAL;           // el càlcul comú, més la
neteja
    }

    @Override
    public String toString() {
        return piscina ? super.toString() + " · piscina privada" : super.toString();
    }
}
```

`Habitacio` passa a ser `extends Allotjament`: perd el codi, el nom i les places, però es queda la regla de les 4 places, que comprova al seu constructor després de `super(...)`. El seu `toString()` és `super.toString()` més « · amb vistes». A `Reserva`, l'habitació passa a ser un allotjament:

```
private final Allotjament allotjament;                          // abans: Habitacio habitacio

public double preu() {
    return allotjament.preuEstada(nits());                      // abans:
habitacio.preuNit() * nits()
}
```

`ProvaPart2` i `ProvaPart3` continuen escrivint el mateix sense tocar-les: hi passen habitacions i suites a `Reserva`, i les habitacions i les suites són allotjaments.

4.3. La sortida és la de l'enunciat. Una setmana a Es Pou Vell costa 7 × 220 + 60 = 1600 €.

4.4.

a) Una classe abstracta no es pot instanciar: quan s'executa `preuEstada()`, l'objecte és per força una habitació, una suite o una casa, i totes tenen `preuNit()`, perquè el compilador les hi obliga. Quin `preuNit()` s'executa es tria en temps d'execució, pel tipus de l'objecte.

b) Perquè una casa paga també la neteja, i `preuNit() * nits()` no la sumaria: 4 nits a Es Pou Vell costarien 880 € en lloc de 940 €. Per arreglar-ho, `Reserva` hauria de saber si l'allotjament és una casa (un `instanceof`). Amb `preuEstada()`, cada allotjament calcula el preu de l'estada a la seva manera, i `Reserva` no ha de saber de quin tipus és.

c) *Casa is not abstract and does not override abstract method preuNit() in Allotjament*. Una classe concreta ha d'implementar tots els mètodes abstractes que hereta. Si no ho fa, també ha de ser abstracta.

4.5.

	Compila?	Què passa, o per què no
a)	**No**	*Allotjament is abstract; cannot be instantiated.*
b)	**Sí**	Una casa és un allotjament.
c)	**Sí**	Escriu `120.0` (4 places × 30 €): el `preuNit()` de `Casa`. Compila perquè `Allotjament` declara `preuNit()`, encara que sigui abstracte.
d)	**No**	*cannot find symbol*: `Allotjament` no té `tePiscina()`. Només el té `Casa`, i la variable és `Allotjament`.
e)	**Sí**	Escriu `420.0`: 3 × 120 + 60 de neteja.

5 Interfícies: la factura

5.1. Una **interfície**. Les reserves i les activitats no comparteixen cap dada ni cap codi: només una capacitat, «es pot facturar», que és un «sap fer» i no un «és un». A més, una classe només pot heretar d'una classe, però pot implementar tantes interfícies com calgui. Si `Facturable` fos una classe abstracta, `Casa` no podria ser mai facturable, perquè ja hereta d'`Allotjament`. Amb una interfície, sí: `public class Casa extends Allotjament implements Facturable`.

5.2.

```
public interface Facturable {
    int IVA_REDUIT = 10;                    // constants: són public static final
    int IVA_GENERAL = 21;

    String concepte();                      // el contracte: public abstract
    double baseImposable();
    int percentatgeIva();

    default double iva() {                 // una sola vegada, i només amb el contracte
        return baseImposable() * percentatgeIva() / 100;
    }

    default double total() {
        return baseImposable() + iva();
    }
}
```

```
public class Activitat implements Facturable {
    private final String nom;
    private final double preuPersona;
    private final int persones;

    // constructor amb les comprovacions, i getters

    @Override
    public String concepte() {
        return nom + " (" + persones + " pers.)";
    }

    @Override
    public double baseImposable() {
        return preuPersona * persones;
    }

    @Override
    public int percentatgeIva() {
        return IVA_GENERAL;                  // la constant de la interfície
    }
    // iva() i total() li arriben fets
}
```

`Reserva` passa a ser `public class Reserva implements Facturable`: el concepte és `"Estada a " + allotjament.getNom() + " (" + nits() + " nits)"`, la base imposable és `preu()` i el percentatge és `IVA_REDUIT`.

```
public class Factura {
    private final String numero;
    private final Client client;
    private final List<Facturable> linies = new ArrayList<>();

    // constructor, getNumero() i getClient()

    public void afegir(Facturable linia) {
        if (linia == null) {
            throw new IllegalArgumentException("La línia no pot ser null");
        }
        linies.add(linia);
    }

    public List<Facturable> getLinies() {
        return Collections.unmodifiableList(linies);
    }

    public double total() {
        double suma = 0;
        for (Facturable linia : linies) {
            suma += linia.total();           // polimorfisme: no sap què és cada línia
        }
        return suma;
    }
}
```

A `Factura` no hi surten mai `Reserva` ni `Activitat`. Si demà es volen facturar els consums del bar, només cal una classe nova que implementi `Facturable`.

5.3. La sortida és la de l'enunciat. Ull amb `baseImposable() * (percentatgeIva() / 100)`: és una divisió entera (10 / 100 = 0), i l'IVA sortiria 0.

5.4.

a) No. Una interfície no pot tenir atributs d'objecte: tot el que s'hi declara com a atribut és una constant (`public static final`). Per això el percentatge és un **mètode**, que cada classe implementa.

b) `iva()` i `total()` es poden calcular **només amb el contracte** (`baseImposable()` i `percentatgeIva()`), i el càlcul és igual per a tothom: poden ser `default`. `percentatgeIva()` depèn de cada classe, i no hi ha cap resposta general: ha de ser abstracte. És la mateixa idea que `preuEstada()` i `preuNit()` a `Allotjament`.

c) Una: la constant `IVA_GENERAL` de `Facturable`. `Activitat` la fa servir pel nom, i no escriu el 21 enlloc.

5.5.

	Compila?	Què passa, o per què no
a)	**No**	*Facturable is abstract; cannot be instantiated*. Una interfície no es pot instanciar.
b)	**Sí**	Una reserva és `Facturable`: la variable de la interfície pot guardar qualsevol objecte que la implementi.
c)	**Sí**	Escriu `1034.0` (940 + 10 %). És el mètode `default`, que crida el `baseImposable()` de la reserva.
d)	**No**	*cannot find symbol*. L'objecte és una reserva, però la variable és `Facturable`, que només té els mètodes del contracte.
e)	**No**	*cannot assign a value to static final variable IVA_GENERAL*. Els «atributs» d'una interfície són constants.

6 Genericitat: els registres i els informes

6.1.

```
public class Registre<T extends Identificable> {
```

Un sol paràmetre de tipus, `T`, el dels elements. D'un `T` qualsevol, el compilador només coneix els mètodes d'`Object`, i `element.getCodi()` no compilaria. Amb la **cota** `T extends Identificable`, sap que tot `T` té `getCodi()`. A més, la cota limita què s'hi pot guardar: un `Registre<Client>` no compila.

6.2. `Allotjament` i `Reserva` ja tenien `getCodi()`: només cal afegir-hi `implements Identificable` i `@Override`. `Habitacio`, `Suite` i `Casa` **no s'han de tocar**: hereten d'`Allotjament` la interfície i la seva implementació. `Reserva` queda amb dues interfícies: `implements Facturable, Identificable`.

```
public interface Identificable {
    String getCodi();
}
```

```
public class Registre<T extends Identificable> {
    private final List<T> elements = new ArrayList<>();

    public void afegir(T element) {
        if (element == null) {
            throw new IllegalArgumentException("No es pot afegir null");
        }
        if (cercar(element.getCodi()) != null) {        // getCodi() compila gràcies a la
cota
            throw new IllegalArgumentException("Ja hi ha un element amb el codi " +
element.getCodi());
        }
        elements.add(element);
    }

    public T cercar(String codi) {                      // retorna un T: no cal càsting
        for (T element : elements) {
            if (element.getCodi().equals(codi)) {     // equals, i no ==: són String
                return element;
            }
        }
        return null;
    }

    public List<T> tots() {
        return Collections.unmodifiableList(elements);
    }

    public int quants() {
        return elements.size();
    }
}
```

```
public class Informes {

    public static double totalFacturat(List<? extends Facturable> elements) {
        double suma = 0;
        for (Facturable element : elements) {          // llegir: tots són Facturable
            suma += element.total();
        }
        return suma;
    }

    public static <T extends Facturable> T mesCar(List<T> elements) {
        T mesCar = null;
        for (T element : elements) {
            if (mesCar == null || element.total() > mesCar.total()) {
                mesCar = element;
            }
        }
        return mesCar;
    }
}
```

6.3. La sortida és la de l'enunciat. El total de les reserves és 346,50 + 632,50 + 1034 = 2013 €. Si compareu els codis amb `==` en lloc d'`equals`, la prova peta amb una `NullPointerException`: el codi `"R002"` es crea amb `String.format` i no és el mateix objecte que el `"R002"` del `main`.

6.4.

a) Les dues crides a `totalFacturat`: amb `reserves.tots()` (*incompatible types: List<Reserva> cannot be converted to List<Facturable>*) i amb `activitats` (el mateix, amb `List<Activitat>`). Una reserva és facturable, però **una llista de reserves no és una llista de facturables**: si ho fos, el mètode podria afegir una activitat dins una llista de reserves. El comodí `? extends Facturable` vol dir «una llista d'algun tipus facturable»: hi caben totes, a canvi de no poder-hi afegir res.

b) Perquè `mesCar` **retorna** un element, i ha de ser del mateix tipus que els de la llista. Amb un comodí retornaria un `Facturable`, i `Reserva r = Informes.mesCar(reserves.tots());` no compilaria sense càsting. El `T` lliga el tipus de la llista amb el del retorn. `totalFacturat` només llegeix la llista i retorna un `double`: n'hi ha prou amb el comodí.

6.5.

	Compila?	Què passa, o per què no
a)	**No**	*type argument Client is not within bounds of type-variable T*. `Client` no és `Identificable`: la cota ho impedeix.
b)	**Sí**	Una suite és una habitació. El registre passa a tenir-ne una.
c)	**No**	*incompatible types: Casa cannot be converted to Habitacio*. El registre és d'habitacions, i una casa no ho és.
d)	**No**	*incompatible types: Allotjament cannot be converted to Casa*. `cercar` retorna un `T`, que aquí és `Allotjament`. Per tenir una `Casa` caldria un càsting.
e)	**Sí**	Però llança `UnsupportedOperationException`: `tots()` és de només lectura. El compilador no ho sap, perquè `List` té `add`.